Universidad Carlos III

Curso Sistemas Distribuidos 2025-26

Práctica 2

Curso 2025-26

Ejercicio 3:

RPC

Fecha: **22/04/2026**

GRUPO: **82 & 84**

Alumnos: **Victor Sainz Mora 100495888**
/ David Torres Paillacho 100522187

Tabla de Contenido

1. DISEÑO REALIZADO..... 2
2. ESTRUCTURA DE FICHEROS..... 2
3. FORMA DE COMPILAR Y EJECUTAR..... 3
4. BATERÍA DE PRUEBAS..... 3

1. DISEÑO REALIZADO

Para el desarrollo de este proyecto hemos implementado un servicio de almacenamiento de tuplas clave-valor accesible de forma remota mediante RPC. El sistema se compone de tres elementos principales: una biblioteca dinámica que actúa como interfaz del cliente, un aplicación cliente que usa es biblioteca y el servidor.

Los ficheros que hemos implementado, es **claves.c**, en el cual mantenemos la estructura de lista enlazada protegida con mutex de hilos para garantizar la concurrencia. Por otro lado, tenemos proxy-rpc.c, en el que implementamos las funciones dentro de claves.h, la cual establece una conexión tcp con el servidor y realiza las llamadas RPC correspondientes. Por último, tenemos server-rpc.c que contiene las funciones **_svc** que el servidor ejecuta al recibir cada petición remota, traduciendo los argumentos RPC al formato interno y delegando en las funciones de claves.c

Además, app-cliente.c es la aplicación que realiza todas las operaciones del servicio, incluyendo pruebas con y sin error con parámetros de entradas erróneos, explicado en el punto 4.
Finalmente, tenemos el fichero .x, escrito en lenguaje XDR, donde definimos la interfaz del servicio RPC, estructuras de datos que se transmiten por la red y las seis operaciones a desarrollar. Gracias a ello y al comando **rpcgen -aNM clavesRPC.x,** se generan automáticamente los ficheros descritos a continuación, que proporcionan toda la infraestructura de comunicación.

2. ESTRUCTURA DE FICHEROS

El proyecto se organiza en los siguiente ficheros principales:

- **clavesRPC.x :** es el fichero de definición de la interfaz RPC en lenguaje XDR. Gracias a este fichero se generan de forma automática el resto de la estructura de comunicación de nuestro proyecto.
- **proxy-rpc.c:** es el fichero donde implementamos las funciones de **claves.h** y hace las llamadas RPC al servidor (conexión entre el cliente y la red).
- **server-rpc.c:** es el fichero que contiene las funciones **_svc** que implementa el servidor. En él, llamamos internamente a las funciones reales de **claves.c**
- **clavesRPC.h:** es la cabecera generada por el comando rpcgen, que contiene las definiciones de tipos y prototipos usados por cliente y servidor.
- **clavesRPC_xdr.c:** son las rutinas de serialización y deserialización generadas por rpcgen
- **clavesRPC_clnt.c:** es el stub del cliente generado por rpcgen, que implementa la parte de la llamada remota en el lado del cliente.
- **clavesRPC_svc.c:** es el stub del servidor generado por rpcgen, que registra el servicio y despacha las peticiones entrantes.
- **claves.c /claves.h:** contiene la implementación de la lógica del servicio, compilada como biblioteca dinámica
- **libclaves.so:** biblioteca dinámica generada al compilar, que encapsula las llamadas RPC del lado del cliente
- **app-cliente.c:** es el fichero que invoca las operaciones remotas del servicio a través de la biblioteca dinámica
- **Makefile:** fichero de construcción que automatiza la compilación y enlazado de todos los componentes.

3. FORMA DE COMPILAR Y EJECUTAR

Primero, a partir del archivo clavesRPC.x, se generan los ficheros que usa RPC c. Para ello se corre el comando: rpcgen -aNM clavesRPC.x. Esto crea varios archivos, como clavesRPC.h, los stubs del cliente y del servidor (clavesRPC_clnt.c y clavesRPC_svc.c) y el fichero clavesRPC_xdr.c.

Después de eso, la compilación se hace ejecutando simplemente "make" en una terminal linux. Al hacerlo, se generan tres archivos. El primero es la biblioteca dinámica libclaves.so, que es la que usa el cliente para hacer las llamadas al servicio. El segundo es el ejecutable del cliente, obtenido de app-cliente.c y enlazado con la biblioteca anterior. Y el tercero es el ejecutable del servidor, llamado clavesRPC_server.

Si se quieren borrar los archivos generados al compilar, se utiliza el comando "make clean".

Para ejecutar el servidor, se usa la orden: "./clavesRPC_server" en una terminal. Para ejecutar el cliente, se abre otra terminal y se definer la variable de entorno IP_TUPLAS, que indica la dirección IP de la máquina donde está corriendo el servidor. Por último se ejecuta el cliente con "./app-cliente".

4. BATERÍA DE PRUEBAS

Esta sección contiene todos los posibles escenarios que nos podemos encontrar en esta práctica, así como, errores y validaciones correctas.

Prueba	**Entrada**	**Salida esperada**	**Salida final**
destroy() inicial		0	0
set_value clave1	key = "clave1", value1 = "hola", N=3, V2=[1.0, 0.33, 30.14], value3 = (5,10, 0)	0	0
set_value clave2	key="clave2", value1="adios", N=2, V2=[2.3,0.5], value3=(7,8,9)	0	0
set_value duplicada	key="clave1" (ya existe)	-1	-1
exist clave existente	key="clave1"	1	1
exist clave inexistente	key="claveX"	0	0
get_value clave1	key="clave1"	value1='hola', N=3, V2=[1.00,0.33,30.14], value3=(5,10,0)	value1='hola', N=3, V2=[1.00, 0.33, 30.14], value3=(5, 10, 0)
get_value ckave2	key="clave2"	value1='adios', N=2, V2=[2.30,0.50], value3=(7,8,9)	value1='adios', N=2, V2=[2.30, 0.50], value3=(7, 8, 9)

modify_value clave1	key="clave1", value1="modificado", N=4, V2=[0.02,99.99,7.7,28.28], value3=(150,200,1200)	0	0
get_value clave1 modificada	key="clave1"	value1='modificado’, N=4, V2=[0.02,99.99,7.70, 28.28], value3=(150,200,1200)	value1='modificado', N=4, V2=[0.02, 99.99, 7.70, 28.28], value3=(150, 200, 1200)
set_value key > 255	key de 256 chars	-1	-1
set_value value1>255	value1 de 256 chars	-1	-1
set_value N=0		-1	-1
set_value N=33		-1	-1
set_value límite exacto	key=255 chars, value1=255 chars	0	0
modify_value clave inexistente	key="inventada"	-1	-1
get_value inexistente	key="inventada"	ERROR	ERROR
set_value key vacía	key = “”	-1	-1
set_value value1 vacío	key = “”	-1	-1
set_value N=1	N_value2=1	0	0
set_value N=32	N_value2=32	0	0
delete_key clave2	key="clave2"	0	0
exist clave2 tras borrar	key="clave2"	0	0
delete_key clave2 repetida	key="clave2" (ya borrada	-1	-1
destroy () final		0	0